

Exercícios – Spring Data JPA (OneToOne e ManyToMany)

Cenário Geral

Os alunos devem criar uma API REST com Spring Boot, JPA e banco de dados em memória (H2).

Deverão usar:

- Controller
- Service
- Repository
- DTO (opcional, caso já usem)

Todos os exercícios devem ser testados via Postman, Insomnia ou Swagger.

Exercício 1 – Pessoa e Documento (OneToOne)

Descrição

Crie um sistema de cadastro onde toda Pessoa possua **um único Documento**, e cada Documento pertença exclusivamente a uma Pessoa.

Entidades

- **Pessoa**
 - id
 - nome
 - documento (OneToOne)

- **Documento**

- id
- numero
- tipo (ex.: RG, CPF)

Requisitos

1. Criar as tabelas com relacionamento @OneToOne.
2. Criar endpoints para:
 - Cadastrar uma pessoa já com documento.
 - Buscar pessoa por id retornando também os dados do documento.
3. Validar que:
 - Uma pessoa não pode ter mais de um documento.
 - Um documento não pode pertencer a mais de uma pessoa.

Testes esperados

- Criar pessoa com corpo JSON:

```
{
  "nome": "Maria Oliveira",
  "documento": {
    "numero": "123456789",
    "tipo": "CPF"
  }
}
```

- Buscar /pessoas/1 deve retornar o documento junto.
-



Exercício 2 – Funcionário e Endereço

Descrição

Agora o relacionamento deve ser bidirecional, ou seja:

- Um Funcionário conhece seu Endereço
- O Endereço conhece seu Funcionário

Entidades

- Funcionário
 - id
 - nome
 - endereco (OneToOne, mappedBy)
- Endereço
 - id
 - rua
 - numero
 - funcionario (OneToOne)

Requisitos

1. Criar relacionamento bidirecional sem estourar erro de serialização (usar DTO).
2. Criar endpoint para cadastrar funcionário com endereço.
3. Endpoint para buscar funcionário e mostrar o endereço.

Objetivo educacional

- Entender ownership (**mappedBy**)
- Resolver problemas de recursão infinita no JSON

Exercício 3 – Curso e Aluno (ManyToMany)

Descrição

Um aluno pode participar de vários cursos, e um curso pode ter vários alunos.

Entidades

- **Aluno**
 - id
 - nome
 - cursos (ManyToMany)
- **Curso**
 - id
 - nome
 - alunos (ManyToMany)

Requisitos

1. Relacionamento ManyToMany utilizando tabela associativa.
2. Endpoints:
 - Criar aluno
 - Criar curso
 - Matricular um aluno em um curso (PUT ou POST)
 - Buscar aluno retornando os cursos
 - Buscar curso retornando seus alunos

Bônus (opcional)

Criar validação para:

- Não permitir vincular o mesmo aluno ao mesmo curso duas vezes.
-

Exercício 4 – Playlist e Música (ManyToMany com dados pré-existent)

Descrição

Os alunos devem salvar músicas primeiro, e depois criar playlists que podem ter várias músicas.

Entidades

- **Musica**
 - id
 - nome
 - artista
- **Playlist**
 - id
 - nome
 - musicas (ManyToMany)

Requisitos

1. Endpoints para:
 - Criar música
 - Criar playlist vazia

- Adicionar músicas existentes na playlist (usando ids)
- 2. Endpoint para buscar uma playlist com todas as músicas.

Desafio extra

Ao excluir uma playlist:

- Não excluir as músicas do sistema (delete cascade apenas na tabela relacional).
-

Exercício 5 – Sistema de Biblioteca (Mix OneToOne + ManyToMany)

Descrição

Agora um sistema maior combinando ambos os relacionamentos.

Entidades

- **Livro**
 - id
 - titulo
 - autor
- **Usuario**
 - id
 - nome
 - contato (OneToOne)
 - livrosEmprestados (ManyToMany)
- **Contato**
 - id

- email
- telefone

Requisitos

1. Criar usuário com seu contato junto.
2. Criar livros.
3. Endpoint para emprestar um livro para um usuário.
4. Endpoint para consultar um usuário e listar:
 - seus dados pessoais
 - contato
 - livros emprestados

Objetivos que eles irão treinar

- Integração entre múltiplos relacionamentos
- Composição de JSON
- Carregamento LAZY (sem estourar erro)